

容斥原理

Hex

容斥原理

利用减法原理，我们可以解决对集合中对象个数的间接计数比直接计数更容易的情况，例如：

- 计数1~n的排列中，1不在第一个位置上的排列数量。

$$(n-1)(n-1)!$$

- 计数1到600之间不能被6整除的整数个数。

$$500$$

减法原理是容斥原理的最简单示例。

容斥原理

设A和B是集合S中分别具有性质P1和P2的集合则有：

$$|\bar{A} \cap \bar{B}| = |S| - |A| - |B| + |A \cap B|$$

$$|A \cup B| = |A| + |B| - |A \cap B|$$

- 计数1到100之间是3或5倍数的整数个数。

$$100/3 + 100/5 - 100/15 = 47$$

- 10个同学选修古代诗歌，6个同学选修现代诗歌，3个同学同时选修了古代诗歌和现代诗歌，问共有多少学生选修诗歌。

$$|A \cup B| = |A| + |B| - |A \cap B| = 10 + 6 - 3 = 13；\text{只选修古代诗歌7人，只选修现代诗歌3人，} 7+3+3=13$$

- 班上有10个人，其中会骑车的有6人，会游泳的有5人，即会骑车又会有用的有3人，问有多少人两种都不会。

$$|\bar{A} \cap \bar{B}| = |S| - |A| - |B| + |A \cap B| = 10 - 6 - 5 + 3 = 2；\text{只会骑车3人，只会游泳2人，} 10-3-2-3=2$$

容斥原理

证明： $|\bar{A} \cap \bar{B}| = |S| - |A| - |B| + |A \cap B|$

对于性质P1和性质P2，有四种可能：既不具有性质P1又不具有性质P2；只有性质P1；只有性质P2；既有性质P1又有性质P2。

只需要证明一个既不具有性质P1又不具有性质P2的对象都给等式右边净贡献1，而其他每一个对象给等式右边净贡献0。

- 既不具有性质P1又不具有性质P2： $1 - 0 - 0 + 0 = 1$
- 只有性质P1： $1 - 1 - 0 + 0 = 0$
- 只有性质P2： $1 - 0 - 1 + 0 = 0$
- 既有性质P1又有性质P2： $1 - 1 - 1 + 1 = 0$

容斥原理

我们可以把关联两个性质的容斥原理扩展到关联任意多个性质的容斥原理。设 $P_1, P_2, \dots, P_m$ 是 S 的对象所设计的 m 个性质，并设

$$A_i = \{x: x \text{ 属于 } S \text{ 且 } x \text{ 具有性质 } P_i\} (i = 1, 2, \dots, m)$$

是 S 的具有性质 P_i (也可能还具有其他一些性质)的对象构成的子集。 $A_i \cap A_j \cap A_k$ 是同时具有性质 P_i, P_j, P_k 的对象的子集。

以此类推，不具备任何性质的对象形成的子集为 $\overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_m}$ 。

容斥原理就是通过计数具有性质的对象来计数上面这个集合中的对象个数。

$$|\overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_m}| = |S| - \sum |A_i| + \sum |A_i \cap A_j| - \sum |A_i \cap A_j \cap A_k| + \dots + (-1)^m |A_1 \cap A_2 \cap \dots \cap A_m|$$

写一写 $m=3$ 的情况， $m=4$ 有多少项？

容斥原理

$$|\overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_m}| = |S| - \sum |A_i| + \sum |A_i \cap A_j| - \sum |A_i \cap A_j \cap A_k| + \cdots + (-1)^m |A_1 \cap A_2 \cap \cdots \cap A_m|$$

证明:

不具备任何性质的对象对这个等式右边的净贡献是1, 至少具备一条性质的对象净贡献是0。

- 设对象 x 不具有任何一条性质: $1 - 0 + 0 - 0 + \cdots + (-1)^m 0 = 1$
- 恰好有 $n \geq 1$ 条性质的对象 y .
 - y 对 $|S|$ 的贡献是 $1 = \binom{n}{0}$ 。
 - 对 $\sum |A_i|$ 的贡献是 $n = \binom{n}{1}$
 - 对 $\sum |A_i \cap A_j|$ 的贡献是 $\binom{n}{2}$
 - ...
 - 即 $\binom{n}{0} - \binom{n}{1} + \binom{n}{2} - \binom{n}{3} + \cdots + (-1)^n \binom{n}{n} = (1 - 1)^n = 0$

容斥原理

$$|A_1 \cup A_2 \cup \dots \cup A_m| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots + (-1)^{m+1} |A_1 \cap A_2 \cap \dots \cap A_m|$$

证明:

$$|A_1 \cup A_2 \cup \dots \cup A_m| = |S| - |\overline{A_1 \cup A_2 \cup \dots \cup A_m}|$$

$$|A_1 \cup A_2 \cup \dots \cup A_m| = |S| - |\overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_m}|$$

例子: 求1到1000之间不能被5, 6和8整除的整数个数。

$$1000 - (1000/5 + 1000/6 + 1000/8) + (1000/30 + 1000/40 + 1000/24) - (1000/120) = 600$$

容斥原理

例子：求字母 M,A,T,H,I,S,F,U,N有多少种排列使得单词，MATH,IS,FUN都不作为连续字母出现在排列中？(INUMATHSF和HTAMISNUF都是不允许的，NUFSIHTAM是允许的)

把集合S当作给定9个字母的所有排列的集合。P1是包含MATH作为连续单词的性质；P2是包含IS作为连续单词的性质；P3是包含FUN作为连续单词的性质。

$$|S| = 9! = 362880$$

A_1 可以看作6个符号{MATH,I,S,F,U,N}的排列，因此 $|A_1| = 6! = 720$

A_2 可以看作8个符号{M,A,T,H,IS,F,U,N}的排列，因此 $|A_2| = 8! = 40320$

A_3 可以看作7个符号{M,A,T,H,I,S,FUN}的排列，因此 $|A_3| = 7! = 5040$

$A_1 \cap A_2$ 可以看作5个符号{MATH,IS,F,U,N}的排列，因此 $|A_1 \cap A_2| = 5! = 120$

$|A_1 \cap A_3| = 4! = 24$ ， $|A_2 \cap A_3| = 6! = 720$ ， $|A_1 \cap A_2 \cap A_3| = 3! = 6$

$$|\overline{A_1} \cap \overline{A_2} \cap \overline{A_3}| = 362880 - 720 - 40320 - 5040 + 120 + 24 + 720 - 6 = 317658$$

容斥原理

例子：求0到99999之间有多少个含有数字2，5和8的整数。

P1:不包含数字2的整数， P2:不包含数字5的整数， P3:不包含数字8的整数。

计算 $|\overline{A_1} \cap \overline{A_2} \cap \overline{A_3}|$

计算 $|A_1 \cap A_2|$ 时，只有数字2和5不能选，其他数字任选，有5个位置可以填，每个位置有8种选择，因此方案数为 8^5 。

$$10^5 - 3 \times 9^5 + 3 \times 8^5 - 7^5$$

P11000 数字串个数

小蓝想要构造出一个长度为 10000 的数字字符串，有以下要求：

小蓝不喜欢数字 0，所以数字字符串中不可以出现 0；

小蓝喜欢数字 3 和 7，所以数字字符串中必须要有 3 和 7 这两个数字。

请问满足题意的数字字符串有多少个？这个数字会很大，你只需要输出其对 10^9+7 取余后的结果。

输出一行一个整数表示答案。

没有0很好解决，只考虑1~9这些数字。

必须有3和7可以枚举所有的3和7的位置，其他位置任意放，但是长度有10000，不可做考虑容斥。

没有数字限制的方案数： 9^{10000}

没有3的方案数和没有7的方案数： 8^{10000}

没有3和7的方案数： 7^{10000}

$$ans = 9^{10000} - 8^{10000} - 8^{10000} + 7^{10000}$$

P15240 解码密钥

给定 k 个正整数 $p_1, p_2, \dots, p_k$ ，定义「有效数字」为能被至少一个 p_i 整除的正整数，求第 n 个有效数字。

举例来说，若 $n = 10$ 、 $k = 3$ 且密钥为 2,3,5。则因为能被 2,3,5 整除的数依次是 2,3,4,5,6,8,9,10,12,14 ...，而第 10 个数是 14，因此所求为 14。

数据范围： $1 \leq n \leq 10^9$, $1 \leq k \leq 6$, $1 \leq p_1 \times p_2 \times \dots \times p_k \leq 10^{18}$ 。

保证所求答案 $\leq 10^{18}$ 。

先考虑怎么拿满前三个点的分

子任务	分数	额外输入限制
1	2	保证所求答案 $\leq 10^6$ 。
2	27	$n \leq 10, k \leq 2$ 。
3	34	$n \leq 10^5$ 。
4	37	无额外限制。

P15240 解

前三个点的分可以通过枚举质数倍数方式解决。
用一个优先队列存储每个质数当前处理到的倍数，
最初的 k 的数为 $pi * 1$ 。
当弹出最小的数 $pi * t$ 时，需要把 $pi * (t + 1)$ 插入
进队列。
相同的数只能算一次。

Unaccepted

63

```
int n, k;
// 定义类型, pii 代表 pair<ll,int> 相当于一个存了两个数的结构体
typedef pair<ll,int> pii;
// 小根堆
priority_queue<pii, vector<pii>, greater<pii>> q;
int main(){
    scanf("%d %d",&n,&k);
    for(int i=1,x;i<=k;i++){
        scanf("%d",&x);
        q.push(make_pair(x, 1));
    }
    for(int i=1;i<=n;i++){
        pii p = q.top();
        // 相同的数要同时出队, 不然会算两次
        while( q.top().first == p.first ){
            pii tmp = q.top();
            // val = p*cnt, 出队后下一次就是 p*(cnt+1)
            ll val = tmp.first, cnt = tmp.second, p = val/cnt;
            q.pop();
            q.push(make_pair(p*(cnt+1), cnt+1));
        }
        // 输出答案
        if( i == n ) printf("%lld\n", p.first);
    }
    return 0;
}
```

P15240 解码密

由于 n 很大，直接枚举肯定不行，由于答案在 10^{18} 以内，考虑二分答案。

每次 $\text{check}(\text{mid})$ 需要计算出 $\leq \text{mid}$ 的有效数字个数。

变成了熟悉的容斥问题，我们可以 2^k 解决。

求 mid 以内， $p_i \times p_j \times p_k$ 的倍数大小为： $\left\lfloor \frac{\text{mid}}{p_i \times p_j \times p_k} \right\rfloor$ 。

总时间复杂度： $O(\log 10^{18} \times k 2^k)$

```
int n, k, p[10];
bool check(ll x){
    ll num = 0;
    // 二进制枚举 若k=3, 且 i = 101(2进制), 代表选择p1*p3
    for(int i=1;i<(1<<k);i++){
        int cnt = 0;
        ll t = 1;
        // 枚举 i 二进制下为1的位置
        for(int j=0;j<k;j++){
            if( i>>j&1 ) {
                cnt++;
                // pi不一定是质数, 需要求lcm
                t = t/gcd(t, p[j+1])*p[j+1];
            }
        }
        // 奇数系数为1, 偶数为-1
        if( cnt&1 ) num += x/t;
        else num -= x/t;
    }
    return num >= n;
}

int main(){
    scanf("%d %d",&n,&k);
    for(int i=1;i<=k;i++) scanf("%d",&p[i]);
    // r = 1e18 可能会有奇怪的精度误差
    ll l = 1, r = 1000000000000000000ll, ans = 0;
    while( l <= r ){
        ll mid = (l+r)/2;
        if( check(mid) ) ans = mid, r = mid-1;
        else l = mid+1;
    }
    printf("%lld\n", ans);
    return 0;
}
```

带重复的组合

n 个不同元素的集合的 r 子集数目(n 个数中选出 r 个数的方案): $\binom{n}{r} = \frac{n!}{r!(n-r)!}$ 。

k 种不同对象, 且每种对象都有无限个的多重集合的 r 组合的个数等于 $\binom{r+k-1}{r}$

- $x_1 + x_2 + \cdots + x_k = r$ 的正整数解数量。

求多重集合 $T = \{3 \cdot a, 4 \cdot b, 5 \cdot c\}$ 的10组合数目。(3个a, 4个b, 5个c中选10个字母的方案数)

也可以理解为 $x_1 + x_2 + x_3 = 10$ ($0 \leq x_1 \leq 3, 0 \leq x_2 \leq 4, 0 \leq x_3 \leq 5$) 的整数解数量。

提示: 从 $T^* = \{\infty \cdot a, \infty \cdot b, \infty \cdot c\}$ 的10组合数目出发, 考虑容斥求解。

带重复的组合

求多重集合 $T = \{3 \cdot a, 4 \cdot b, 5 \cdot c\}$ 的10组合数目？

设 $T^* = \{\infty \cdot a, \infty \cdot b, \infty \cdot c\}$ 的10组合集合为 S

设 P_1 是 T^* 的10组合中 a 出现 **多于3次** 的性质，设 P_2 是 T^* 的10组合中 b 出现 **多于4次** 的性质，

设 P_3 是 T^* 的10组合中 c 出现 **多于5次** 的性质。

答案为 $|\overline{A_1} \cap \overline{A_2} \cap \overline{A_3}|$ ，根据容斥原理：

$$|\overline{A_1} \cap \overline{A_2} \cap \overline{A_3}| = |S| - (|A_1| + |A_2| + |A_3|) + (|A_1 \cap A_2| + |A_1 \cap A_3| + |A_2 \cap A_3|) - |A_1 \cap A_2 \cap A_3|$$

如何计算 $|A_1|$ ？由于 a 出现多于三次，相当与必选4个 a ， $T^* = \{\infty \cdot a, \infty \cdot b, \infty \cdot c\}$ 的 **6** 组合数目 $\binom{6+3-1}{6}$ 。

依此类推得到每一项，最后答案为 6。

带重复的组合

满足 $1 \leq x_1 \leq 5$, $-2 \leq x_2 \leq 4$, $0 \leq x_3 \leq 5$ 的方程 $x_1 + x_2 + x_3 = 10$ 的整数解数量？

令 $y_1 = x_1 - 1$, $y_2 = x_2 + 2$, $y_3 = x_3$, 方程变为 $y_1 + y_2 + y_3 = 11$ 。

条件变为: $0 \leq y_1 \leq 4$, $0 \leq y_2 \leq 6$, $0 \leq x_3 \leq 5$ 。

.....

容斥求解即可，答案15。

P1450 [HAOI2008] 硬币购物

共有 4 种硬币。面值分别为 c_1, c_2, c_3, c_4 。

某人去商店买东西，去了 n 次，对于每次购买，他带了 d_i 枚 i 种硬币，想购买 s 的价值的东西。请问每次有多少种付款方法。

输入格式

输入的第一行是五个整数，分别代表 c_1, c_2, c_3, c_4, n 。

接下来 n 行，每行有五个整数，描述一次购买，分别代表 d_1, d_2, d_3, d_4, s 。

输出格式

对于每次购买，输出一行一个整数代表答案。

$1 \leq c_i, d_i, s \leq 10^5, 1 \leq n \leq 1000$ 。

P1450 [HAOI2008] 硬币购物

直接多重背包： $O(nds) = O(10^{13})$

如果没有硬币数量限制，如何解决？

- 完全背包预处理， $O(1)$ 输出结果。

```
for(int i = 1; i <= 4; ++i)
    for(int j = c[i]; j <= MAXN; ++j)
        dp[j] += dp[j - c[i]];
```

考虑有一种硬币有限制，那么 $dp[s] - dp[s - c[i] * (d[i] + 1)]$ 就是方案数。

- 红色部分的意思就是，把超过限制的方案数减去，相当于强制使用了 $d[i] + 1$ 个硬币。

有多种硬币怎么办？

- 先把4个硬币单独限制的方案数减去。
- 这时发现两种硬币限制的方案减重复了，要加回去。
- 三种硬币限制的方案加多了，要减回去。
-

类似的题目P13349

错排

错位排列是指没有任何元素出现在它原本位置的排列。长度为 n 的错排数目记作 D_n 。

长度为3的错排有：[2,3,1], [3,1,2]

找规律：

$$D_1 = 0, D_2 = 1, D_3 = 2, D_4 = 9, D_5 = 44, D_6 = 265 \dots$$

观察： $2 \times 4 + 1 = 9$, $9 \times 5 - 1 = 44 \dots$

验证： $0 \times 2 + 1 = 1$, $1 \times 3 - 1 = 2 \dots$

猜测：

$$D_n = n \times D_{n-1} + (-1)^n$$

错排

直接证明 $D_n = n \times D_{n-1} + (-1)^n$ 非常麻烦，先尝试从排列组合的意义推出错排公式。

- 提示：长度为 n 的错排如何从长度小的错排推出。

用 n 与长度 $n-1$ 错排的任意一个元素交换，都能得到一个长度为 n 的错排。 $(n-1) \times D_{n-1}$

在长度 $n-1$ 排列中令 $a_i = i$ ，其他 $n-2$ 个元素错位排列，第 i 个元素与 n 交换。 $(n-1) \times D_{n-2}$

$$D_n = (n-1)(D_{n-2} + D_{n-1})$$

如何推出 $D_n = n \times D_{n-1} + (-1)^n$?

错排

$$D_n = (n-1)(D_{n-2} + D_{n-1})$$

$$D_n - nD_{n-1} = -(D_{n-1} - (n-1)D_{n-2})$$

$$\text{令 } F_n = D_n - nD_{n-1}$$

$$F_n = -F_{n-1}$$

$$F_2 = D_2 - 2D_1 = 1$$

$$F_n = (-1)^n$$

$$\text{因此 } D_n = nD_{n-1} + (-1)^n$$

错排

通项公式:

$$D_n = n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!} \right)$$

设 S 是 $\{1, 2, \dots, n\}$ 的全部 $n!$ 个排列的集合。对 $j = 1, 2, \dots, n$, 设 P_j 是一个排列中 j 在第 j 位的性质即 $a_j = j$ 。 $\{1, 2, \dots, n\}$ 的一个排列是错位排列当且仅当它不具有性质 $P_1, P_2, \dots, P_n$ 中的每一条性质。设 A_j 是具备 P_j 性质的排列的集合, $D_n = |\overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n}|$ 。

有 $|A_1 \cap A_2 \cap \cdots \cap A_k| = (n - k)!$, 从 n 中选 k 个的方案数有 $\binom{n}{k}$ 个。

$$D_n = n! - \binom{n}{1} (n - 1)! + \binom{n}{2} (n - 2)! - \binom{n}{3} (n - 3)! + \cdots + (-1)^n \binom{n}{n} 0!$$

$$D_n = n! - \frac{n!}{1!} + \frac{n!}{2!} - \frac{n!}{3!} + \cdots + (-1)^n \frac{n!}{n!}$$

P4071 [SDOI2016] 排列计数

求有多少种1到 n 的排列 a ，满足序列恰好有 m 个位置 i ，使得 $a_i = i$ 。答案对 $10^9 + 7$ 取模。

输入 #1

```
5
1 0
1 1
5 2
100 50
10000 5000
```

输出 #1

```
0
1
20
578028887
60695423
```

测试点编号	$T =$	$n, m \leq$	测试点编号	$T =$	$n, m \leq$
1 ~ 3	10^3	8	10 ~ 12	10^3	10^3
4 ~ 6	10^3	12	13 ~ 14	5×10^5	10^3
7 ~ 9	10^3	100	15 ~ 20	5×10^5	10^6

对于全部的测试点，保证 $1 \leq T \leq 5 \times 10^5$ ， $1 \leq n \leq 10^6$ ， $0 \leq m \leq 10^6$ 。

答案为： $\binom{n}{m} D_{n-m}$ ，只需要预处理阶乘及逆元和错排即可 $O(1)$ 查询。